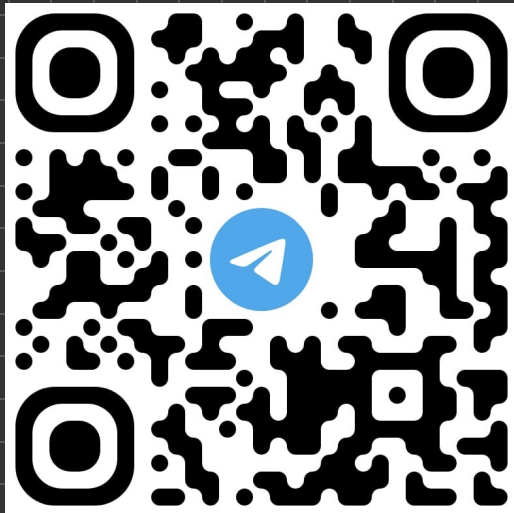


JS
Javascript

Programming language

Telegram



Discord



Language

Laptop / computer → machine

- right click
- new folder
- name
- create button

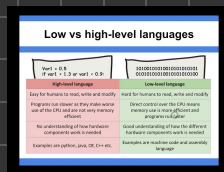
} Instructions

low level vs High level programming language

machine

binary code

~~0101100110110~~



High-level languages

high

```
Var1 = 0.5  
if var1 > 1.3 or var1 < 0.9:
```

low

```
10100101010010101010101  
01010101010010101010100
```

High-level language

Easy for humans to read, write

Programs run slower as they use more of the CPU and are not very efficient

No understanding of how hardware components work is needed

Examples are python, java, C#, C++ etc.

High Level Language

```
if (var1)  
{  
    print("I am true block")  
}  
else  
{  
    print("I am else block")  
}
```

VS

Low Level Language

```
100000  
1000000001100  
1100100010000100  
10000000011000100  
11001101101101100011
```

Low-level language

Hard to read, write and modify

Programs run faster for the CPU means they are more efficient and run faster

Good understanding of how the different hardware components work is needed

Examples are machine code and assembly language

High Level

Language

```
if (i<5)
{
    print("I am true block");
}
else
{
    print("I am else block");
}
```

VS

Low Level

Language

```
110100
1000000001100
110010001000100
10000000011000100
11001110111011100011
```



Compiled ✓

(C++, Java)

vs

interpreted language

(JS, python)

high
level

source code

→ 100 lines

translation → compiler

machine (0 & 1)

execute

low
level

fast

source code
translation → 100 lines

execute

execution line by line

slow

Why JS? ✓

html, css, JS

→ has modern web browser

Frontend → ReactJS

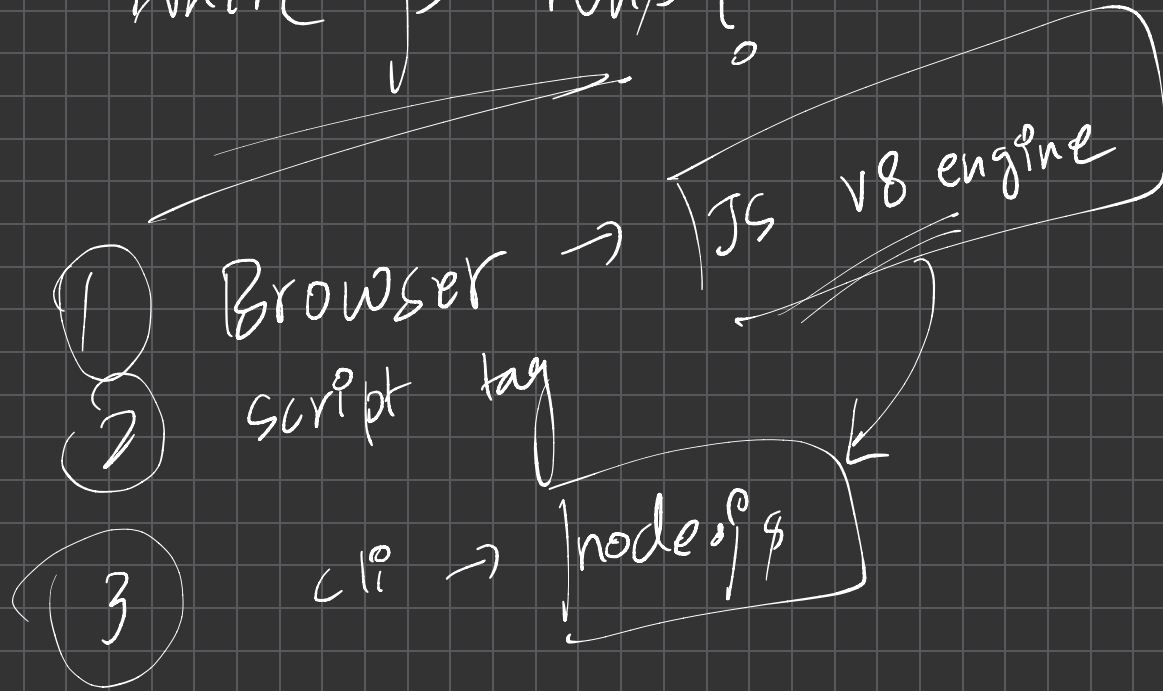
Backend → node.js

Logically
working

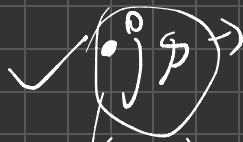
cost of 1
thread

Add to cart

Where js runs?



✓ html ← html code

✓  javascript code
extension

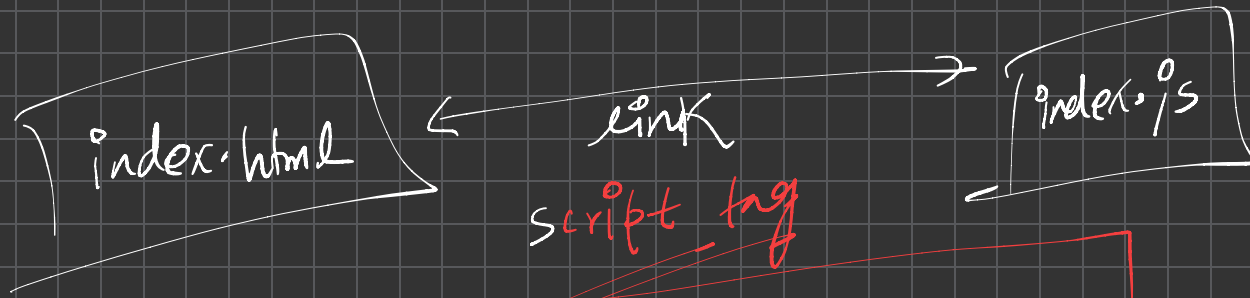
✓ lecture 29 : js foundation

 index.html

 index.js

vs code

Javascript file hai



script tag

`<script src="" ></script>`

↓
source

!.

→ Head

→ Body

JS

JS

avacomp / lecture 2 / js foundation / 10 mark

1 console.log("Hello World");

→
syntax

↓
print

↓
string

console.log ($5+6$)
 ↓
 terminal

String
'Nishant'

Math
 $5+6$

→ { "5+6"
 '5+6' }

$$4 * 5 = 20$$

mark

using "4 * 5" = "string 4 * 5")

CSS ←

⇒ Variables ?

box

age → 19

box name → age = 19 → box value

Comment in JS
= //

③ keyword → used to declare variables

↳ var, let & const

scope

→ declaration
→ initialization }

✓ `var age;` // declaration → box created without value

✓ `age = 24` // initialization → box mai value de di

`var age = 24` // both together declaration and initialization

Var ~~X~~

```
var age = 23;  
console.log(age)  
var age = 24; // re-declaration ✓  
age = 26 // updation ✓  
console.log(age)
```

let ✓

```
let name = "nishant";  
console.log(name);  
let name = "saini"; // re-declaration not allowed ✗  
console.log(name);
```

```
/Users/nishantsaini/Desktop/web dev earners/javascript/lecture 29 - js foundation/index.js  
let name = "saini";  
^
```

SyntaxError: Identifier 'name' has already been declared
at wrapSafe (node:internal/modules/cjs/loader:1004:16)
at Module._compile (node:internal/modules/cjs/loader:1845:20)
at Object..js (node:internal/modules/cjs/loader:2002:10)
at Module.load (node:internal/modules/cjs/loader:1594:32)
at Module._load (node:internal/modules/cjs/loader:1396:12)
at wrapModuleLoad (node:internal/modules/cjs/loader:255:19)
at Module.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:154:5)
at node:internal/main/run_main_module:33:47

```
let name = "nishant";  
console.log(name);  
// let name = "saini"; // re-declaration not allowed ✗  
name = "saini" // updation allowed ✓  
console.log(name);  
|
```

const

```
// ✓  
const aadharCard = 9879862345689;  
console.log(aadharCard);  
// const aadharCard = 74656354463; // re-declaration not allowed ✗  
aadharCard = 62546246464; // updation not allowed ✗
```



```
aadharCard = 62546246464;  
^
```

TypeError: Assignment to constant variable.

```
at Object.<anonymous> (/Users/nishantsaini/Desktop/Web dev earners/JavaScript/lecture 29 : js foundati  
on/index.js:36:12)  
at Module._compile (node:internal/modules/cjs/loader:1871:14)  
at Object..js (node:internal/modules/cjs/loader:2002:10)  
at Module.load (node:internal/modules/cjs/loader:1594:32)  
at Module._load (node:internal/modules/cjs/loader:1396:12)  
at wrapModuleLoad (node:internal/modules/cjs/loader:255:19)  
at Module.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:154:5)  
at node:internal/main/run_main_module:33:47
```

Variable Naming rules

→ meaning full variable name

→ variable name must start with either a letter or an underscore or dollar \$ sign

→ Reserved keywords ko
bhi use nhi kr skte
as a variable name

_
\$
@
+

conventions

↓ JS

① →

camelCase → Nishant Saini

Aadhar Card Number
nishantSaini

aadharCardNumber

② snake-case → python

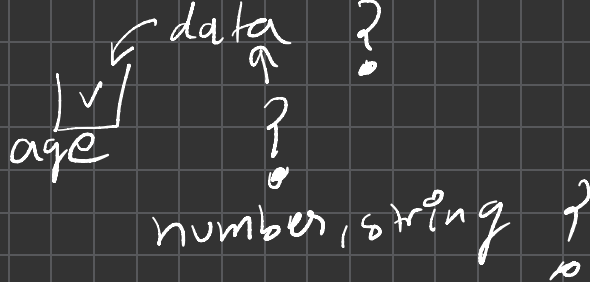
first_Name
↳ first_name - - -

③ Pascal Case
↓

④ SCREAMING_SNAKE ✓

DB-URL

Data types



Primitive

- number
- string
- boolean
- null
- undefined
- BigInt } x
- symbol }

Non-Primitive (complex)

- object
- Array
- function

number → 45, 65, 10, 9.5, 3.1, 6.9, -1, -5
integer, decimal, negative

string → sequence of characters

"hello world"

'hello world'

hello world → template literal

boolean → true | false

bug

null → khali? → intentionally

undefined → value he nhi di (TS)

data type check karne ke liye
↓
typeof

```
let collegeName = "London Academy"
let mobileNumk const userName = "Ramu"
const greetingMessage = 'Hi!, ${userName}'
console.log(greetingMessage); 54
console.log(typeof collegeName);
console.log(typeof mobileNumber);
console.log(typeof(mobileNumber));
```

Template literal

→ Backticks

→ `${}`

↑ variables
JS

```
const userName = "Manu"  
const greetingMessage = `Hi, ${userName}`  
console.log(greetingMessage);
```